

MEGA PROJECT: Microservices CI/CD (Updated Ports)

Overview

This is a capstone-level Mega Project designed to demonstrate a highly complex, multi-service architecture in a single CI/CD pipeline.

You will orchestrate three distinct microservices written in three different languages, build them in parallel, scan them, and deploy them using Docker Compose.

Port Architecture (Conflict-Free)

Since your Jenkins is locked to port `8080` and SonarQube is locked to `9000`, the applications have been explicitly mapped to avoid conflicts:

1. **Frontend UI (Python Flask):** Acts as the API Gateway/Dashboard. Maps to port `3000`.
2. **Product Service (Java Spring Boot):** Manages inventory data. Maps to port `8081`.
3. **Order Service (Node.js/Express):** Manages user orders. Maps to port `8082`.
4. **Orchestrator:** `docker-compose.yml` binds these services together on the `jenkins-net` network.

Prerequisites

Ensure the following are installed and running on your Jenkins server:

- Docker & Docker Compose (`docker compose` plugin)
- Trivy
- Python 3 & pip
- Java 17 & Maven
- Node.js & npm

Step 1: Network Setup

Ensure your shared network exists:

```
docker network create jenkins-net
```

Step 2: Infrastructure Setup

Run SonarQube and Jenkins:

```
# Run SonarQube
```

```
docker run -d --name sonarqube -p 9000:9000 --network jenkins-net  
sonarqube:lts-community
```

```
# Run Jenkins (Using your existing 8080 port)
```

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 --network  
jenkins-net -v /var/run/docker.sock:/var/run/docker.sock -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

Step 3: Jenkins Configuration

1. Log into Jenkins at `http://localhost:8080`.
2. Install necessary plugins: **Docker Pipeline**, **SonarQube Scanner**, **Pipeline**.
3. Configure SonarQube Server in Manage Jenkins -> System.
4. Ensure global tools (Maven, Node, Sonar Scanner) are accessible.

Step 4: Pipeline Execution (The Magic)

1. Create a new Pipeline job.
2. Provide the `Jenkinsfile`.
3. Run the Build!

Step 5: Verification

Access the master dashboard which aggregates data from the other microservices over the internal Docker network:

- URL: `http://localhost:3000`

- Expected Output:

```
{"dashboard":"Mega E-Commerce","orders":[{"id":1,"item":"Laptop","status":"Shipped"}],"products":["Laptop","Smartphone","Headphones"]}
```